CV-Lab6 卷积神经网络实验报告

杨子涵　学号：2400012732

2025 年 12 月 12 日

1　任务 1：数据增强 (Data Augmentation)

1.1　实现思想

为了提高模型在 CIFAR-10 数据集上的泛化能力，防止过拟合，在训练阶段引入了数据增强策略。我主要采用了以下两种几何变换以及标准化的预处理方式：

1. **随机裁剪 (Random Crop)**：首先对原始 32×32 的图像四周进行 4 像素的 Padding（填充），使图像变为 40×40。随后在其中随机裁剪出一个 32×32 的区域。这模拟了物体在视场中位置的平移变化。

2. **随机水平翻转 (Random Horizontal Flip)**：以 $p = 0.5$ 的概率对图像进行水平翻转，利用图像的左右对称性扩充数据。

3. **标准化 (Normalization)**：使用 CIFAR-10 数据集的统计均值 $(0.4914, 0.4822, 0.4465)$ 和标准差 $(0.2023, 0.1994, 0.2010)$ 对图像通道进行归一化，加速模型收敛。

1.2　算法流程

算法 1 数据增强预处理流程

输入: 原始训练图像 $I \in \mathbb{R}^{32\times32\times3}$

输出: 增强后的张量 $T_{out} \in \mathbb{R}^{3\times32\times32}$

$I_{pad} \leftarrow \text{Padding}(I, \text{padding} = 4)$

$I_{crop} \leftarrow \text{RandomCrop}(I_{pad}, \text{size} = 32)$

$I_{flip} \leftarrow \text{RandomHorizontalFlip}(I_{crop}, \text{prob} = 0.5)$

$T \leftarrow \text{ToTensor}(I_{flip})$

$T_{out} \leftarrow \text{Normalize}(T, \text{mean}, \text{std})$

return T_{out}

2 任务 2：VGG 网络实现

2.1 实现思想

参考经典的 VGG 网络架构，构建了一个深度的卷积神经网络。本实验实现了一个包含 **5 个卷积块 (Convolutional Blocks)** 和 **3 层全连接分类器** 的结构。

网络架构细节：

- **特征提取部分：**
  - Block 1 & 2：各包含 2 个卷积层，通道数分别为 64 和 128。
  - Block 3, 4 & 5：各包含 3 个卷积层，通道数分别为 256, 512, 512。
  - 每个卷积层均为 3×3 卷积，步长为 1，padding 为 1，保持特征图尺寸不变。
  - 每个卷积层后紧接 **Batch Normalization** 层和 **ReLU** 激活函数。
  - 每个卷积块末尾接一个 2×2 的最大池化层 (Max Pooling)，将特征图尺寸减半。
- **分类器部分：**
  - 将特征图展平 (Flatten) 后，连接两层 512 维的全连接层和一层输出为 10 的全连接层。
  - 全连接层之间引入了 **Dropout** ($p = 0.5$)，随机丢弃神经元以抑制过拟合。

2.2 VGG 前向传播伪代码

算法 2 VGG 网络结构

输入: 图像 Batch x

```
function BLOCK(x, num_convs)
    for i = 1 to num_convs do
        x ← ReLU(BatchNorm(Conv3x3(x)))
    end for
    x ← MaxPool2x2(x)
    return x
end function

x ← Block(x, 2)                                   ▷ 64 channels
x ← Block(x, 2)                                   ▷ 128 channels
x ← Block(x, 3)                                   ▷ 256 channels
x ← Block(x, 3)                                   ▷ 512 channels
x ← Block(x, 3)                                   ▷ 512 channels
x ← Flatten(x)
x ← Dropout(ReLU(Linear(x, 512)))
x ← Dropout(ReLU(Linear(x, 512)))
y ← Linear(x, 10)
return y
```

3 任务 3：ResNet 网络实现

3.1 实现思想

ResNet 引入了残差学习 (Residual Learning) 框架，解决了深层网络的退化问题。

1. **残差块 (ResBlock)**：核心公式为 $y = \sigma(F(x, \{W_i\}) + x)$。本实验代码中实现的 Block 包含主分支和 shortcut 分支：
   - 主分支：$\text{Conv}(3 \times 3) \to \text{BN} \to \text{ReLU} \to \text{Conv}(3 \times 3) \to \text{BN}$。
   - Shortcut：当输入输出通道数不一致或步长不为 1 时，使用 1×1 卷积进行投影；否则直接恒等映射。
2. **网络结构**：包含一个初始卷积层，随后是 4 个 Layer 阶段。根据代码 `make_layer` 的参数，Layer 1 到 Layer 4 的通道数依次为 64, 128, 256, 512，每个 Layer 包含 **2 个残差块**。最后通过全局平均池化 (AdaptiveAvgPool) 和全连接层输出。

3.2 残差块流程

算法 3 ResBlock 前向传播

输入: 特征图 x

Main Path:
$out \leftarrow \text{ReLU}(\text{BatchNorm}(\text{Conv3x3}(x, \text{stride})))$
$out \leftarrow \text{BatchNorm}(\text{Conv3x3}(out, \text{stride} = 1))$
Shortcut Path:
if stride > 1 **or** in_channel $\neq$ out_channel **then**
    $identity \leftarrow \text{BatchNorm}(\text{Conv1x1}(x))$
else
    $identity \leftarrow x$
end if
Output:
$y \leftarrow \text{ReLU}(out + identity)$
return y

4 任务 4：ResNeXt 网络实现

4.1 实现思想

ResNeXt 结合了 VGG 的堆叠思想和 ResNet 的残差思想，并提出了 **Cardinality** (基数) 的概念，利用分组卷积 (Group Convolution) 提升网络容量。

1. **ResNeXt Block**：采用了瓶颈结构 (Bottleneck) 的改进版：
    - 1×1 卷积降维。
    - 3×3 **分组卷积** (Group Conv)：代码中设置 `groups=32`。
    - 1×1 卷积升维。
    - 残差连接同 ResNet。
2. **网络结构**：网络整体框架与 ResNet 类似。根据代码，每个 Stage 堆叠了 **3 个 ResNeXt Block**，通道数随层级加倍。

4.2 ResNeXt Block 流程

算法 4 ResNeXt Block 前向传播

输入: 特征图 x, Cardinality $C = 32$

Main Path (Split-Transform-Merge):

$out \leftarrow \mathrm{ReLU}(\mathrm{BatchNorm}(\mathrm{Conv1x1}(x)))$ ▷ 降维

$out \leftarrow \mathrm{ReLU}(\mathrm{BatchNorm}(\mathrm{GroupConv3x3}(out, \mathrm{groups} = C)))$ ▷ 分组卷积

$out \leftarrow \mathrm{BatchNorm}(\mathrm{Conv1x1}(out))$ ▷ 升维

Shortcut Path:

同 ResBlock，若维度不匹配则使用 1×1 卷积调整。

Output:

$y \leftarrow \mathrm{ReLU}(out + shortcut)$

return y

5 实验结果与分析

基于上述网络结构和超参数设置，在 TensorBoard 上记录了训练过程中的 Loss 和 Accuracy 变化曲线。

5.1 实验曲线展示

按照如上网络结构和超参数训练得到的模型在 TensorBoard 上记录得到的 Test Acc、Test Loss、Train Loss、Train Acc、Learning Rate 曲线如下。

5.1.1 VGG

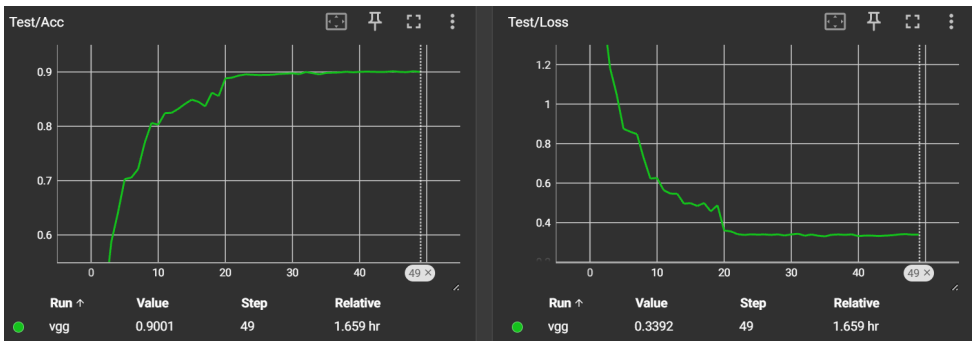

图 1: VGG 的 Test/Acc 与 Test/Loss 曲线

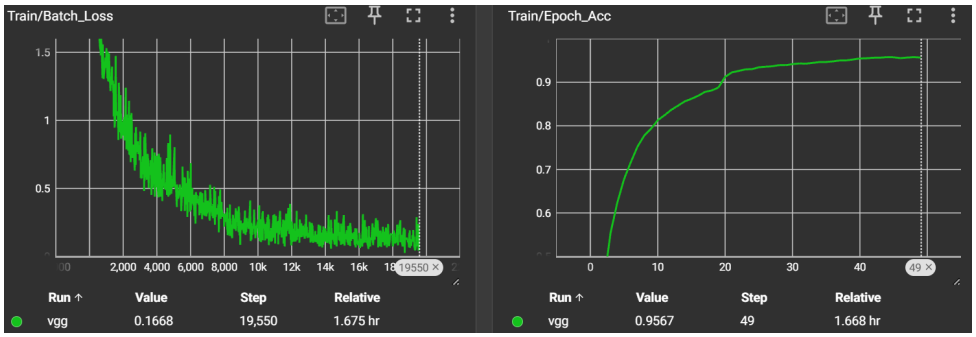

图 2: VGG 的 Train/Batch_Loss 与 Train/Epoch_Acc 曲线

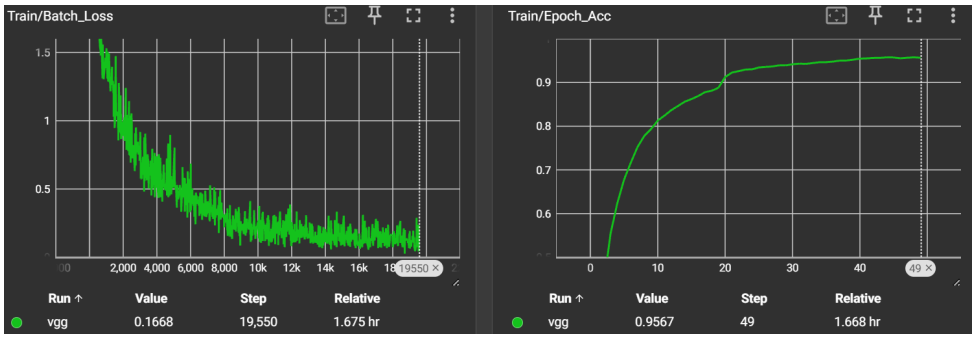

图 3: VGG 的 Train/Epoch_Loss 与 Train/Learning_Rate 曲线

5.1.2 ResNet

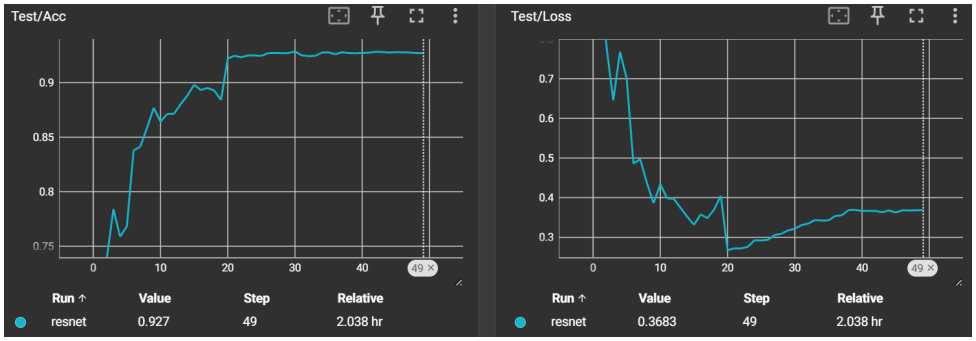

图 4: ResNet 的 Test/Acc 与 Test/Loss 曲线

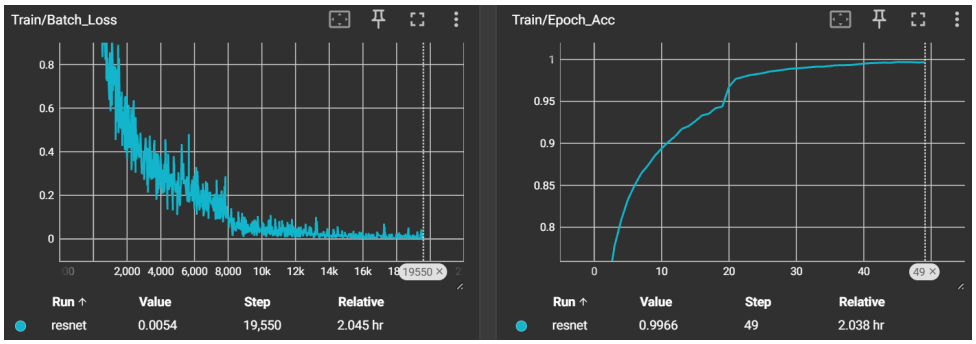

图 5: ResNet 的 Train/Batch_Loss 与 Train/Epoch_Acc 曲线

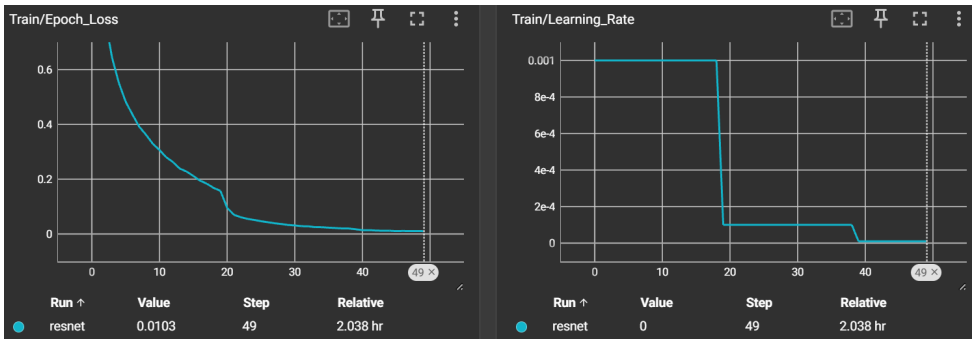

图 6: ResNet 的 Train/Epoch_Loss 与 Train/Learning_Rate 曲线

5.1.3 ResNeXt

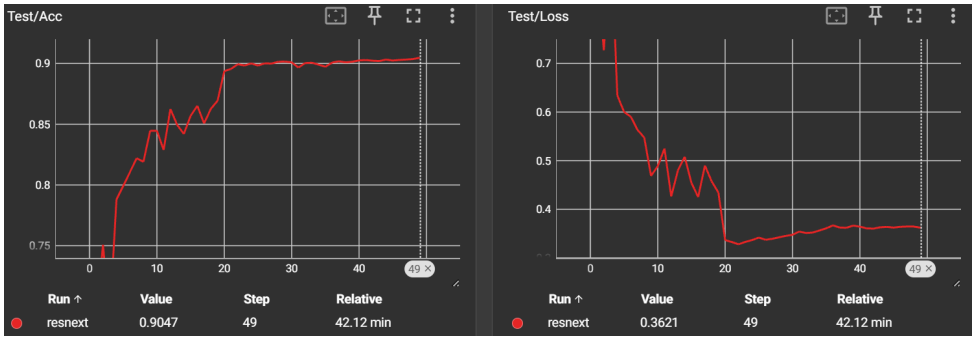

图 7: ResNeXt 的 Test/Acc 与 Test/Loss 曲线

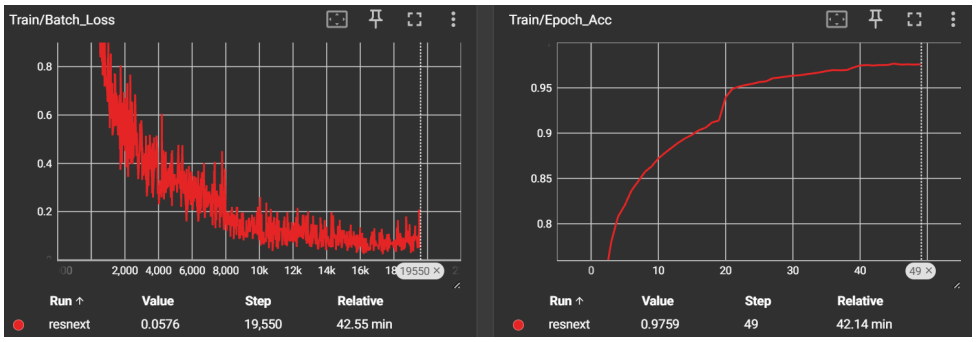

图 8: ResNeXt 的 Train/Batch_Loss 与 Train/Epoch_Acc 曲线

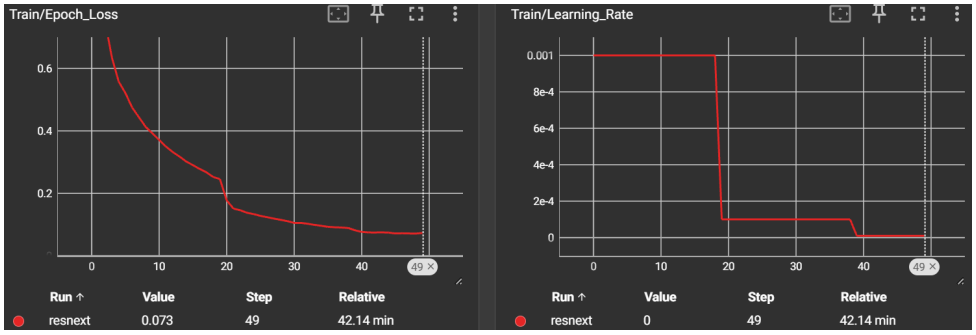

图 9: ResNeXt 的 Train/Epoch_Loss 与 Train/Learning_Rate 曲线

5.2 结果分析

1. **最终正确率对比：**观察曲线可知，VGG在测试集上的准确率达到了90.12%，ResNet在测试集上的准确率达到了92.87%，ResNeXt在测试集上的准确率达到了90.47%。

2. **收敛速度对比：**观察训练曲线可知，ResNet 和 ResNeXt 相比 VGG 收敛更为平稳。这是因为残差连接提供了梯度的“高速公路”，有效缓解了深层网络训练初期的梯度问题。VGG 在训练初期 Loss 下降较快，但波动相对较大。

3. **泛化能力：**在 Test Accuracy 方面，**ResNet** 和 **ResNeXt** 普遍优于 VGG。VGG 虽然参数量巨大，但全连接层较多，容易出现过拟合现象（尽管使用了 Dropout）。而 ResNeXt 通过增加 Cardinality，在不显著增加参数量的情况下，进一步提升了特征提取的丰富度，通常能获得比 ResNet 稍好的最终精度。

4. **数据增强的作用：**训练集 Accuracy 和测试集 Accuracy 之间的差距保持在合理范围内，并未出现严重的过拟合（例如训练集 99% 但测试集仅 70% 的情况）。这证明了 RandomCrop 和 RandomHorizontalFlip 等数据增强手段有效地扩充了数据集分布，提升了模型对物体位移和翻转的不变性。

6 额外说明

用python main.py –model resnet –epochs 20格式的指令可以训练模型，用python main.py –model resnet –action test格式的指令可以加载已有的checkpoint